

JS

function

```
class Date{  
}
```

```
class Employee{  
}
```

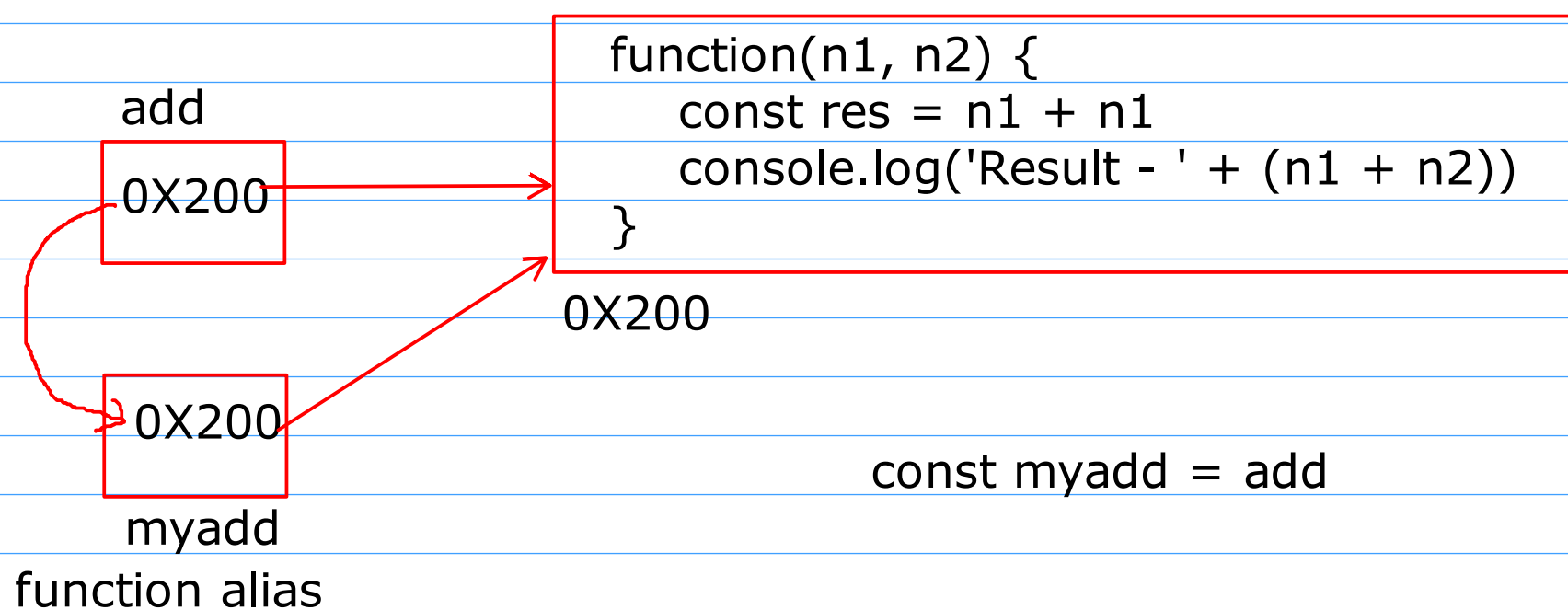
Date d1;

Employee e1;

Object

myadd = 10

function const x = add



closure

- the inner functions maintaining the state of the variables declared inside the outer functions are called as closures

```
void f1(){  
  static int count = 0;  
  count ++  
  printf("%d",count);  
}  
  
f1();  
f1();  
f1();
```

```
class BankAccount{  
  private static int generateAccno;  
  int accno;  
  
  BankAccount(){  
    generateAccno++;  
    this.accno = generateAccno;  
  }  
}
```

int BankAccount::generateAccno = 100;

```
Date d1;?  
  
int main(){  
  Date d1;  
  Date d2;  
  Date d3;  
  
}
```

```
class Date{  
  int day;  
  int month;  
  int year;  
}
```

```
BankAccount a1;  
BankAccount a2;  
BankAccount a3;
```

day	day	day
month	month	month
year	year	year

closures

```
in_fn
() => {
    count = 4
    count++;
    console.log('count - ' + count)
}
```

```
in_fn2
count=1
() => {
    count++;
    console.log('count - ' + count)
}
```

in_fn() // 1
in_fn() // 2
in_fn() // 3
in_fn() // 4

in_fn2() // 1

```
let data
getStudents(){
    data = []
}
data = ()=>{
getStudent()
data.push()
data.remove()
```

currying

C, CPP, Java, Python , JS, C#

Language Fundametal
- Rules
- Syntax

```
class Date{
    int day; //variable Data members -> Fields
    Time t1; // variable Date member object
}

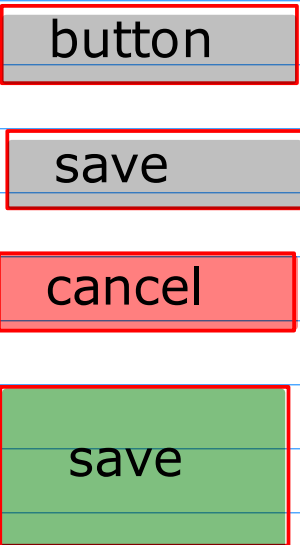
int main(){
int day; // variable
Date d1;// Object
}
```

C-> 10
CPP -> OOP
Python
Java -> syntax, rules, features
Ad java
JS -> 3 days -> 15 hrs

- Creating a sequence of nested functions taking a single paramater

```
function createButton(name='button',color='gray',size=50px){
return <button style="bg-color, size" >name</button>
}

createButton()
createButton('save')
createButton('cancel','red')
createButton('save','green', 100px)
```



Higher Order function

- Its a function that accepts another function into its paramter is called as higher order function

```
function f1(n) {
  console.log(n)
}
f1((n1, n2) => n1 + n2)
```

- # Object
- JSON
1. object template
2. new keyword
3. constructor function

```
class Person{
  string name;
  int age;
  Person(string name=" ", int age=0){
    this->name = name;
    this->age = age;
  }
};

Person *p1 = new Person();
Person *p2 = new Person('anil',32); // dynmic initialization
```

```
void Person(string name=" ", int age=0){
  this->name = name;
  this->age = age;
}
```

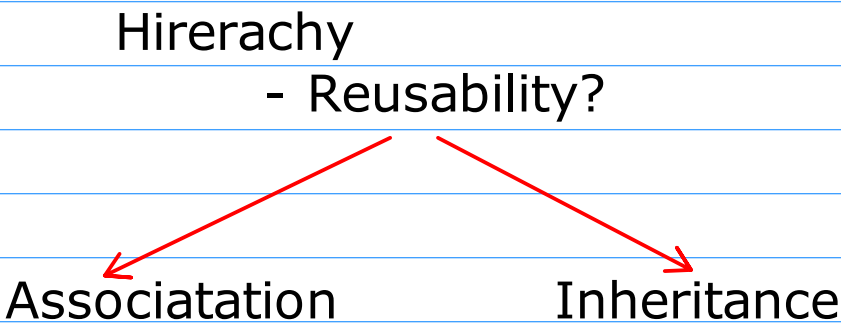
[[prototype]]

```
class Date{
  int day,month,year;
}

class Person{
  string name;
  Date dob;// associatation
}
```

has-a

is-a



```
class Employee :Person{
  Date doj;
  Date dot;
}

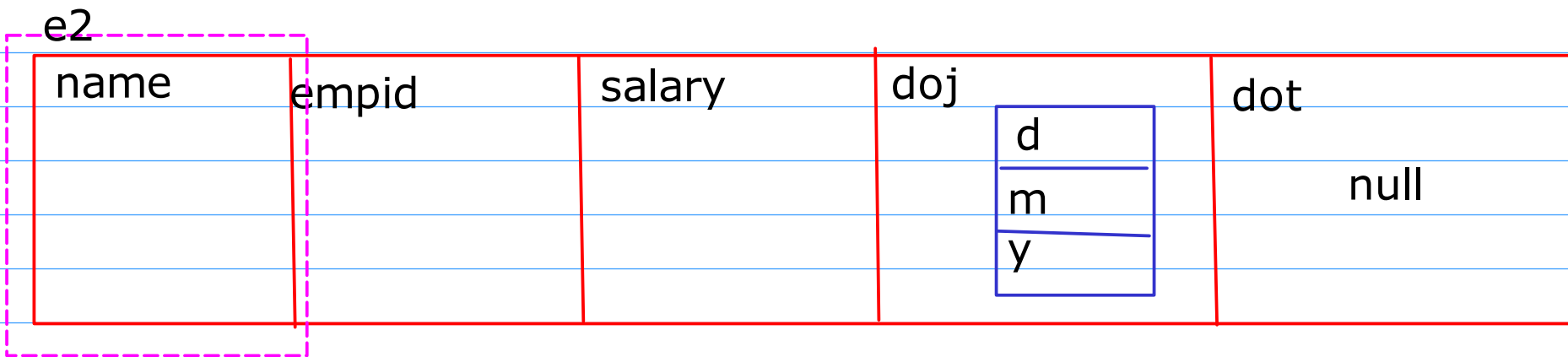
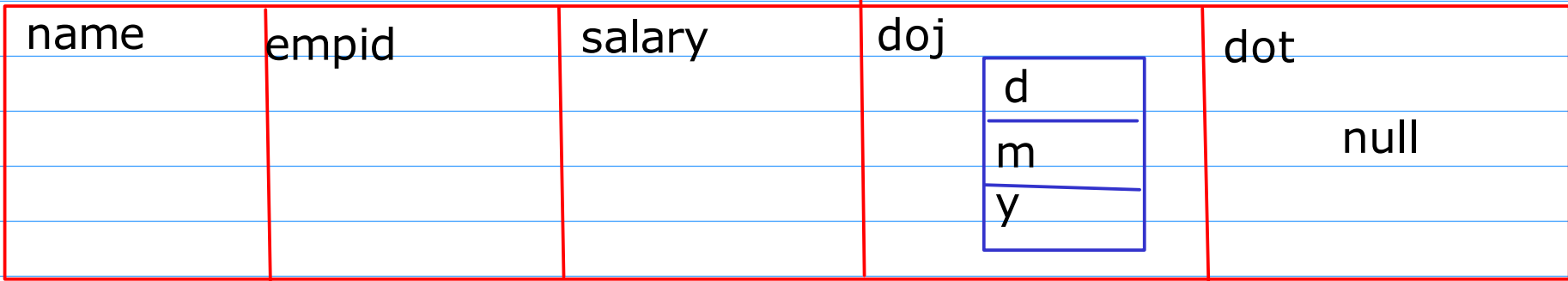
class Product{
  Date dom;
  Date doe;
  Date od;
  Date dd;
}
```

```
class Date{
    int d;
    int m;
    int y;
    void displayDate(){ }
}

class Person{
    string name;
    void displayPerson(){
    }
}

class Employee:Person // inheritance{
    Date doj; // Association
    Date *dot;
    int empid;
    double salary;
}

Employee e1;
```



```
e1.name
e2.name
e1.displayPerson()
e1.doj.d;
e1.doj.displayDate()
```